

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

UNIVERSIDAD TECNOLÓGICA NACIONAL

Bases de Datos II

Trabajo Práctico Integrador

Parte 2: Integración, operaciones CRUD desde aplicación y backups

Dominio de negocio: E-Commerce de Tecnología

Stack: Python + PyMongo (Driver Nativo) + MongoDB Atlas

Integrantes: Sussini Guanziroli Patricio
Vazquez Matias Ezequiel

Año: 2026

Bloque 1 — Operaciones CRUD desde la Aplicación

Gestión de la conexión

La conexión se establece usando PyMongo, el driver nativo oficial de MongoDB para Python. El cliente se instancia una sola vez y se reutiliza para todas las operaciones:

```
from pymongo import MongoClient
from bson import ObjectId
from datetime import datetime, timezone

# — Conexión —
# String de conexión
MONGO_URI = "mongodb+srv://db_admin:admin1234@cluster-techstore.bvseful.mongodb.net/"
DB_NAME   = "techstore_db"

client = MongoClient(MONGO_URI)
db = client[DB_NAME]
productos = db["productos"]

print("Conexión exitosa a MongoDB Atlas - TechStore")
```

Las 4 operaciones:

Operación	Funcion	Descripción
CREATE	crear_producto()	Inserta un producto nuevo. Se crean con activo: True automáticamente.
READ	listar_productos()	Consulta solo productos con activo: True (respeta baja lógica).
UPDATE	actualizar_producto())	Modifica campos específicos con \$set sin reemplazar el documento.
DELETE	dar_de_baja()	Baja lógica: \$set { activo: False }. No se elimina físicamente.

CREATE — Inserción de un nuevo producto

```
# — CREATE —
def crear_producto(nombre, descripcion, categoria_id_str, precio, stock,
especificaciones=None):
    """Inserta un nuevo producto en la colección. Nace con activo: True."""
    nuevo = {
        "nombre": nombre,
        "descripcion": descripcion,
        "categoria_id": ObjectId(categoria_id_str),
        "precio": precio,
        "stock": stock,
        "especificaciones": especificaciones or {},
        "imagenes": [],
        "fecha_alta": datetime.now(timezone.utc),
        "activo": True,
    }
    resultado = productos.insert_one(nuevo)
    print(f"[CREATE] Producto insertado con _id: {resultado.inserted_id}")
    return resultado.inserted_id
```

READ — Listado respetando la baja lógica

```
# — READ —
def listar_productos():
    """
    Devuelve solo los productos con activo: True.
    """
    cursor = productos.find({"activo": True})
    lista = list(cursor)
    print(f"[READ] {len(lista)} productos activos encontrados:")
    for p in lista:
        print(f" - {p['_id']} | {p['nombre']} | ${p['precio']} | stock: {p['stock']}")
    return lista
```

UPDATE — Modificación parcial con \$set

```
# — UPDATE —
def actualizar_producto(producto_id, cambios: dict):
    """Actualiza campos específicos de un producto con $set (no reemplaza el
documento)."""
    resultado = productos.update_one(
        {"_id": ObjectId(producto_id), "activo": True},
        {"$set": cambios}
    )
    if resultado.matched_count:
        print(f"[UPDATE] Producto {producto_id} actualizado. Campos:
{list(cambios.keys())}")
    else:
        print(f"[UPDATE] No se encontró el producto {producto_id}")
    return resultado.matched_count
```

DELETE — Baja lógica (no elimina el documento)

```
# — DELETE (baja lógica)
def dar_de_baja(producto_id):
    """
    Baja lógica: setea activo: False sin borrar el documento físicamente.
    """
    resultado = productos.update_one(
        {"_id": ObjectId(producto_id), "activo": True},
        {"$set": {"activo": False}}
    )
    if resultado.matched_count:
        print(f"[DELETE] Producto {producto_id} dado de baja lógica (activo: False)")
    else:
        print(f"[DELETE] No se encontró el producto {producto_id} (o ya estaba dado de baja)")
    return resultado.matched_count
```

Bloque 2 — Backups y Resguardo

3.1 — Script CLI (backup.sh)

El script crea con rutas relativas la carpeta `./resguardos_tpi/<YYYY-MM-DD>/` y ejecuta `mongodump` conectándose directamente al cluster de Atlas:

```
#!/bin/bash

set -e

MONGO_URI="mongodb+srv://db_admin:admin1234@cluster-techstore.bvseful.mongodb.net/"
DB_NAME="techstore_db"
FECHA_HOY=$(date +%Y-%m-%d)
CARPETA_DESTINO="./resguardos_tpi/${FECHA_HOY}"

echo "====="
echo " Backup de ${DB_NAME}"
echo " Fecha: ${FECHA_HOY}"
echo "====="

mkdir -p "${CARPETA_DESTINO}"
echo "Carpeta creada: ${CARPETA_DESTINO}"

mongodump --uri="${MONGO_URI}" --db="${DB_NAME}" --out="${CARPETA_DESTINO}"

echo "====="
echo " Backup finalizado: ${CARPETA_DESTINO}/${DB_NAME}"
echo "=====
```

3.2 — Análisis RTO / RPO aplicado a TechStore

RPO (Recovery Point Objective) — cuantos datos podemos permitirnos perder.

Con este script ejecutado manualmente, el RPO equivale al tiempo transcurrido desde el último backup ejecutado. Si ocurre un incidente justo antes del próximo dump, se perderían todos los pedidos, altas de clientes y cambios de stock desde ese momento. Para un e-commerce real esto es inaceptable, dado que una venta no respaldada es una venta perdida. Considerando una estrategia productiva, automatizaríamos el script (cron en Linux, Programador de tareas en Windows) con backups diarios completos e incrementales más frecuentes en colecciones de alta escritura como pedidos.

RTO (Recovery Time Objective) — cuanto tiempo puede estar el sistema caído.

Al usar backup completo (no incremental, ni diferencial), el RTO es bajo. Ante un incidente basta con un único mongorestore desde el dump más reciente para volver a un estado consistente, sin necesidad de aplicar una cadena de incrementales. En el cluster M0 con datos de prueba la restauración tarda minutos, en producción con miles de pedidos podría escalar según el tamaño de la base.

Conclusión: la estrategia de este TPI prioriza la simplicidad de restauración (RTO bajo) sobre la frescura de los datos (RPO alto). En producción habría que automatizar la frecuencia para bajar el RPO a un nivel aceptable para el negocio.

Conclusión — Desafíos de la comunicación Cliente-Servidor

Conectar la aplicación a MongoDB Atlas en lugar de trabajar directamente en mongosh expuso diferencias concretas en el flujo de los datos:

- **El driver como intermediario:** PyMongo traduce las llamadas de Python a mensajes de red en el protocolo de MongoDB. Lo que antes era `db.productos.find()` en la shell ahora es `productos.find()` en el código, con la misma semántica pero atravesando la red hasta Atlas en cada operación.
- **Tipos que no viajan igual:** un `ObjectId` de MongoDB no es un string. Al recibir un id como parámetro hay que convertirlo explícitamente con `ObjectId(id_str)` antes de usarlo en una query, de lo contrario MongoDB no encuentra el documento aunque el id visualmente parezca correcto.
- **Baja lógica como contrato de la aplicación:** el campo activo: `True` no es solo un dato, sino que es un filtro que toda función de lectura debe respetar explícitamente. Si `listar_productos()` omitiera el filtro, mostrará productos dados de baja sin ningún error visible, rompiendo silenciosamente la regla de negocio definida respecto a bajas lógicas.
- **Backup desacoplado de la app:** `mongodump` se conecta directamente al cluster sin pasar por el código Python. Esto separa dos responsabilidades, siendo una la aplicación gestionando los datos en tiempo real, y otra, el script de backup como una tarea de administración de infraestructura independiente del código de negocio.
- **Conexión única y cierre explícito:** a diferencia de la shell que abre y cierra la conexión automáticamente, en el driver hay que cerrar el cliente con `client.close()` al terminar. Dejar conexiones abiertas consume recursos del cluster incluso cuando la app ya no hace operaciones.

En síntesis, el mayor aprendizaje fue entender que mediante el driver cada llamada a `productos.insert_one()` o `productos.find()` genera una conexión de red, serializa los datos a BSON, los envía al cluster, espera la respuesta y los deserializa de vuelta a objetos Python. Entender ese flujo es lo que permite diagnosticar errores y optimizar consultas.